KernelCoder Skill 发现与按需加载

Skill 模块负责扫描本地技能、展示完整目录，并在模型选择或用户明确指定后加载正文。目录只包含名称和描述；正文通过 `load_skill` 作为工具结果进入对话。当前实现面向少量本地技能，不引入向量召回、检索重排或额外的路由模型调用。

本文对应 skills.py、Agent Loop、prompt.py、ContextManager 和 CLI。

1. 整条链路

```
启动 CLI
  ↓
SkillLoader.scan：扫描配置目录，解析 SKILL.md
  ├─ 有效技能 → 建立名称、描述、路径、来源和版本索引
  └─ 格式错误、同名覆盖 → 记录 diagnostics
  ↓
Agent 初始化
  ├─ 零技能 → 不注册 load_skill，不添加技能目录
  └─ 有技能 → 注册 load_skill，完整目录只放入 system 一次
  ↓
用户输入
  ├─ 普通任务 → 主模型阅读目录，自行选择零个、一个或多个技能
  │               ↓
  │             模型调用 load_skill(skill="名称")
  └─ /skill NAME [任务] → Agent 在首次模型请求前调度 load_skill
                  ↓
hooks → Permission → 工作区保护 → 读取完整 SKILL.md
                  ↓
工具结果进入历史之前，登记本轮活跃技能
  ├─ 同版本完整正文仍可见 → 复用，新增结果仅记录复用说明
  └─ 正文已裁剪、已移除或版本变化 → 保留本次完整正文
                  ↓
receive → prepare_context → 主模型请求
                  ↓
按技能指引执行；需要时再读取引用资料、调用脚本工具
```

`scan()` 负责发现，主模型负责匹配任务，`load()` 负责提供正文。扫描本身不执行技能、不调用模型，也不自动运行技能目录中的脚本。

2. 修改前后的区别

项目	修改前	当前版本
YAML 描述	按行拆分冒号，无法正确解析多行描述	使用 `yaml.safe_load`，支持 `description: |` 和 `>`
名称	使用目录名	优先使用 frontmatter 的 `name`；与目录名不同会记录提示
目录	单条描述最多 250 字符，总计最多 3,000 字符	展示完整名称和完整描述，不按排序隐藏尾部技能
目录位置	同时存在于工具描述和 system 工具列表	独立放在 system，工具 schema 仅保留固定说明
扫描诊断	多数错误静默忽略	`/skills` 展示解析失败、名称差异和同名覆盖
更新	启动后工具描述可能过期	`/skills reload` 同步更新索引、工具注册和系统提示
显式调用	依赖模型主动调用工具	`/skill NAME [任务]` 在模型请求前完成加载
正文生命周期	普通工具结果，可能被压缩	本轮活跃正文受到四层 compact 保护，下一轮重新选择
重复加载	每次把全文加入历史	相同完整正文仍可见时复用；不使用永久的 loaded 标志

目录全部展示意味着其开销随技能数量和描述长度增长。当前版本不做候选筛选：目录或活跃正文放不下时，沿统一上下文预算路径明确报错，不静默截断。

3. 扫描目录与元数据

默认顺序如下，先发现的同名技能生效：

```
<项目>/.kernelcoder/skills/<目录名>/SKILL.md
~/.kernelcoder/skills/<目录名>/SKILL.md
```

E2B 工作区模式下，项目级目录使用当前任务副本中的 `.kernelcoder/skills`，刷新也针对该副本。用户级目录仍来自本机用户目录。

扫描只检查配置根目录的直接子目录，不递归扫描项目、依赖目录或引用资源。同名覆盖会记录胜出文件与被覆盖文件。符号链接和解析后越出配置根目录的文件会拒绝；加载时再次检查，防止扫描后路径被替换。

字段	用途
`name`	工具参数与 `/skill` 使用的精确名称
`description`	展示给主模型的完整路由描述，多行空白归一化
`path`	已发现的 `SKILL.md` 路径，不接受任意外部路径作为技能名
`source`	所属配置根目录，便于确定来源和检查路径
`fingerprint`	扫描时完整文件的 SHA-256；每次加载另算实际版本

有 YAML frontmatter 时，描述必须是非空字符串；YAML 无法解析、描述类型错误或名称无效时跳过并记录诊断。为兼容原有本地格式，没有 frontmatter 的文件仍可使用第一个一级标题作为描述。

扫描会读取 `SKILL.md` 以解析和计算版本，但注册表不缓存正文。**按需加载指按需进入模型上下文，不表示启动扫描完全不读取文件。** `references/`、`scripts/`、`assets/` 内容不随扫描或正文加载自动读取。

4. 完整目录与任务选择

Agent 的 `_full_messages()` 动态添加 `SKILL_PROMPT` 和 `SkillLoader.summary()`，例如：

```
# Available skills
……技能选择和使用规则……
- code-review: 检查代码变更，重点关注正确性、错误路径和测试。
- pdf: 创建和处理 PDF；当用户要求生成或操作 PDF 文件时使用。
```

`load_skill` 的工具 schema 不重复这份目录。模型通过用户的实际任务判断是否加载，不要求关键词一出现就使用技能。例如"解释 PDF 是什么"可以直接回答；"把这篇文档转成 PDF"才需要对应的操作流程。

当前版本不额外调用分类模型，也不新增 `search_skills`。目录完整展示，主模型直接调用：

```
{"skill": "code-review"}
```

`/skills` 显示目录条数及其估算 token 开销。该开销和正文一起纳入现有 `max_context_tokens` 预算，仍为其他系统提示、工具 schema、模型输出和安全余量留出空间。

5. 加载、版本与显式调用

`SkillLoader.load()` 通过注册表查询名称，再读取文件。返回格式为：

```
Skill: code-review
Version: <本次完整文件的 SHA-256>
Base directory: <技能所在目录>

<完整 SKILL.md，包括 frontmatter>
```

正文单独修改后，下次加载直接读取新内容并返回新的版本。名称或描述修改后，加载会提示 `/skills reload`，避免模型依照旧目录加载含义已经变化的技能。文件删除、无法读取或路径越界时返回错误，不登记活跃状态。

用户显式输入 `/skill code-review 检查本次修改` 时，Agent 保留用户请求原文，创建实际执行的 `load_skill` 调用，并在首次模型请求前写入对应工具结果。此调用同样经过 hooks、Permission 和工作区保护；拒绝或失败时返回结果，不继续请求模型假装已经加载成功。未知名称和缺少名称在修改历史之前返回错误。

Skill 只提供指令。加载技能不会执行其脚本，也不会授予文件写入、Bash 或网络权限；后续操作仍经过原有工具权限与执行边界。技能要求的相对资源以 `Base directory` 解析；沙箱中的执行资源仍须位于可访问的任务副本中，不会因为技能加载而自动同步用户级目录中的脚本。

6. 与 Context Compact 的配合

成功结果经过 `_append_message()` 时，Agent 根据实际工具调用确认其来自内建 `SkillTool`，再调用 `ContextManager.activate_skill()`。登记发生在 `receive()` 之前，防止大正文首次进入就被裁剪。

阶段	对本轮活跃技能的处理
`tool_budget_constraint`	跳过活跃技能正文；同批次其他普通结果仍可按规则生成预览
`snip_compact`	保留活跃正文所在的完整工具批次，允许超过条数软目标
`micro_compact`	不缩短活跃正文
`compact_history` / `fit_tool_results`	快照仍包含它，但不把正文送入摘要，也不把它改成预览

活跃正文与其他不可裁剪内容合计过大时，抛出 `ContextLimitError`，保留完整内容。保护消息不会扩大模型窗口。

本实现将**一次 `Agent.chat()`，即一个用户请求及其后续工具循环**作为技能激活周期。`begin_turn()` 清除上一轮活跃标记；模型若要在新一轮继续使用技能，需要再次加载：

- 相同版本的完整正文仍在历史中：重新激活原消息，新结果只说明复用。
- 旧正文已裁剪或被 snip 删除：重新读取当前文件，提供完整正文。
- 文件内容变化：新正文成为活跃版本，旧版本解除保护。

因此，没有"曾经加载过，以后永远跳过"的缓存。复用比较完整返回内容，包含版本与来源。为省去重复正文而写入的短结果仍保留自己的工具调用配对。

会话通过既有 `context_state` 保存 `kind`、`skill_name` 和 `skill_active` 等消息元数据，恢复后手动 compact 仍能识别活跃正文；新用户请求开始时按上述规则重新选择。旧会话仍可加载。`reset()` 清空历史和激活状态，保留技能目录；子 Agent 拥有独立历史和 ContextManager，不继承父级激活状态。

7. 一组使用示例

当前即使没有安装任何技能，也可以正常启动并对话。零技能时没有技能目录或 `load_skill` schema。

后续添加技能时，可以在项目中创建以下文件；这里仅展示格式，不自动安装样例：

```
.kernelcoder/skills/code-review/SKILL.md
```

```
---
name: code-review
description: >
  检查代码变更中的正确性、错误处理和测试遗漏。
  当用户明确请求审查代码、审查 diff 或检查修改风险时使用。
---
# 代码审查

先阅读相关 diff 和调用位置，再判断问题是否能由代码证据支持。
按严重程度列出可操作问题，给出文件位置和触发条件。
说明实际运行过的验证，不把计划执行的测试写成已通过。
```

```
/skills                  查看现有目录、来源目录、估算开销和诊断
/skills reload           新增、删除或修改元数据后刷新
请审查本次修改            模型根据完整目录自行决定是否调用 load_skill
/skill code-review       显式加载，再由模型响应当前请求
/skill code-review 检查错误处理
/compact                 压缩历史，本轮活跃正文完整保留
/save                    保存消息及技能激活元数据
/reset                   清除会话，下次使用技能重新加载
```

例如，模型先加载 `code-review`，之后读取 50 条工具结果，触发 snip。技能批次继续保留，较早的普通工具结果可以归档。用户下一轮提出新请求时，旧技能解除保护；若它随后被裁剪，再调用 `load_skill` 会提供完整文件，而非返回失效的"已经加载"状态。

8. 实现位置与验证

文件	主要职责
`kernelcoder/skills.py`	YAML 解析、目录扫描、诊断、来源和版本、完整目录、正文加载
`kernelcoder/prompt.py`	集中的技能选择与使用提示
`kernelcoder/agent.py`	注册和刷新工具、目录注入、显式调用、成功结果登记
`kernelcoder/context.py`	活跃正文去重、四层保护、轮次释放和既有状态持久化
`kernelcoder/cli.py`	`/skills`、`/skills reload`、`/skill` 与帮助信息
`kernelcoder/tools/agent.py`	子 Agent 共享可发现目录，激活状态独立
`tests/test_skills.py`	无付费 API 的扫描、调用、压缩、恢复和 CLI 回归测试

测试覆盖多行 YAML、格式错误、同名覆盖、完整目录超过旧字符限制、零技能增删刷新、显式调用与权限拒绝、版本变化、重复加载、并行配对、四层压缩保护、旧正文回读、会话恢复、reset、子 Agent 隔离和超预算明确失败。模型交互使用模拟 LLM；这些测试验证工程行为，不代表真实模型在所有任务上都能正确选中技能。